

How to use a Source DLL and YFan.dll to generate uniform ray fans in non-sequential mode

Authored by

David Keith

Abstract

This article provides an overview of the YFan.dll functionality. While random sources with a large number of rays are generally best for optimization and radiometric analysis, a uniform fan is often useful for visualization. This DLL can be used with a Source DLL in non-sequential mode to provide a uniformly spaced ray fan. The usual position and tilts parameters can be used to reorient a YFan as desired.

Sample files

- YFan.zar demonstrates the use of YFan.dll.

Downloads

- YFan.dll must be placed in the Zemax/DLL/Source directory. This can be accomplished by opening YFan.zar.
- YFan.cpp is the C++ source code.

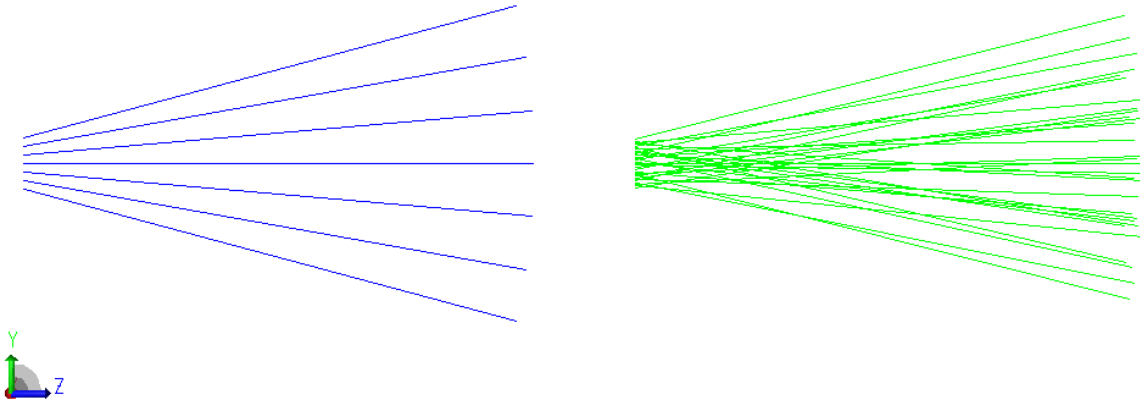
In this article

- Introduction
- Implementation
- Installing YFan.dll
- Instructions for use with a Source DLL object
- Examples

Introduction

For both radiometric analysis (for example determining the power incident on a detector) and optimization, the best choice for a source is almost always a pseudo-random source such as a Source Ellipse, with enough analysis rays to provide for smooth statistics. Frequently, this means tens of thousands to millions of rays. For visualization, OpticStudio provides the capability to designate a lesser number of Layout Rays to appear in layout plots. These layout rays are still randomly distributed. If only a few rays are used, they generally do not well represent the design. If enough rays are used to visualize the extent of the volume occupied by rays, then the plot can become crowded.

On the left, a YFan with 0.5 half-height and 7 rays between ± 15 degrees. On the right, a Source Two-Angle with similar specifications and 30 rays. It still does not indicate the beam width.



One can, of course, generate a ray list to be used with a Source File object. Many users do that with tools like Excel, Matlab, or Mathematica. But this is time consuming, and eventually one winds up with a large number of such files, each with some unique but still forgettable name.

YFan.dll, when used as the data for a Source DLL object, provides the ability to generate a uniform fan of rays where the parameters defining the fan are entries in the Non-sequential Component Editor (NCE).

In this article, we will show how to use YFan.dll in OpticStudio's Non-sequential Mode.

Implementation

Code defining the YFan.dll was written in C++ and compiled with Microsoft Visual Studio 2019. The DLL implements two functions:

- UserParamNames -- receives a pointer to an array. The first element in the array contains an integer indicating the number of the user-defined parameter whose name should be returned by the function. The name is returned by modifying the elements of the same array. This allows the DLL to provide the column headings for the user-defined parameters.
- UserSourceDefinition -- receives a pointer to an array. The array contains the values of the user-defined parameters which have been entered in the NCE. For each call, the function returns the parameters defining a single ray. These are the values for x, y, z, l, m, n and intensity. The values are returned by modifying the passed array.

Notice that the paradigm used for calling the source DLL expects to make a separate call for each ray. This works well if they are served from a random distribution. But in this case, we want to generate and serve a specific set of rays to form an evenly spaced fan. To provide for this a special approach is taken. On first call, and on any call for which the user defined parameters have changed, the complete

ray set is calculated, and a ray counter is reset. Each call then returns the ray designated by the counter and then increments the counter. The parameters passed to UserSourceDefinition do not include “# Layout Rays” so the number of rays in the fan must be defined separately in the user-defined parameters. If the number of layout rays is less than the number of rays in the fan, then only a portion of the fan will be shown. If the number of layout rays is more than the number of rays in the fan, then some rays will be duplicated. (This is also true for analysis rays, if any are defined.)

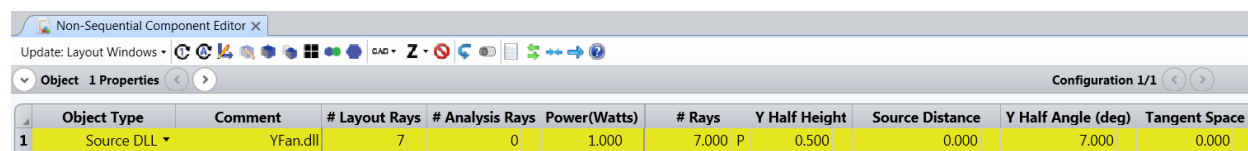
In most cases, the fan will be used for visualization only, and contain only layout rays, and so will contribute zero power to analyses. In this case, a good practice is to define the number of rays in the “# Layout Rays” entry and use a pickup in the “# Rays” entry to control the fan.

Installing YFan.dll

The file YFan.dll should be placed in the Zemax DLL/Sources directory. In the default installation that resides in the user documents directory. Opening the example file YFan.zar will place YFan.dll in the correct directory automatically.

Instructions for use as a Source DLL

Below is a section of the NCE in which a YFan has been defined. (Columns not related to the discussion have been hidden.)



Object Type	Comment	# Layout Rays	# Analysis Rays	Power(Watts)	# Rays	Y Half Height	Source Distance	Y Half Angle (deg)	Tangent Space
1 Source DLL	YFan.dll	7	0	1.000	7.000 P	0.500	0.000	7.000	0.000

- The source has been entered as a Source DLL and YFan.dll selected from the pull-down list.
- The number of rays in the fan has been entered as the # Layout Rays.
- There are zero analysis rays, so the Power of 1 Watt does not affect results.
- The # Rays in the fan implements a pickup from # Layout Rays.
- The Half Height of the is 0.5 lens units.
- With a Source Distance of 0, the Y Half Angle controls the fan. (to be explained)
- The fan distribution is not in tangent space, and so it is in angle space. (to be explained)

Parameters

Four parameters control the generation of rays for the fan:

Rays: Sets the number of rays.

- If less than 2, then 2 rays are generated.
- If greater than 101, then 101 rays are generated.
- #Rays does not control the number of rays for display or analysis.
- If these are fewer, then only a portion of the rays are used.
- If greater, then some rays are duplicated.

Y Half Height: Controls the Y-extent of the fan in position space.

- With respect to the source location, all rays have an origin on the local y-axis.

- Rays will originate from y locations from -"Y Half Width" to +"Y Half Width" with an even spacing.

Source Distance: If non-zero, rays appear to emanate from a point on the z axis at this distance from the location of the source object. In that case, the remaining parameters are ignored.

- A negative value results in diverging rays.
- A positive value results in converging rays.

Y Half Angle: Controls the angular extent of the fan in degrees.

- Rays are directed in the yz plane with an angle with respect to the z-axis between -"Y Half Angle" and +Y "Half Angle".

Tangent Space: Controls the angular distribution of the fan.

- If "Tangent Space" = 0, then the ray angles are evenly spaced.
- If "Tangent Space" != 0, then the tangents of ray angles are evenly spaced. If from a point source, this will result in an even spacing of their y intercepts on a distant wall. If from an extended source with a negative half-angle the rays will converge to a point.

Examples

Figure 1: A non-zero half height and zero half angle produce a collimated beam

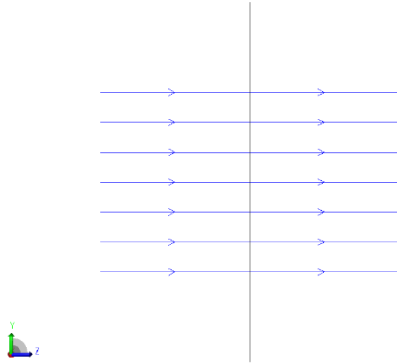


Figure 2: A zero half height fan in angle space

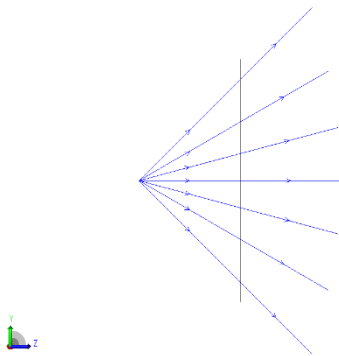


Figure 3: Zero half height fan in tangent space

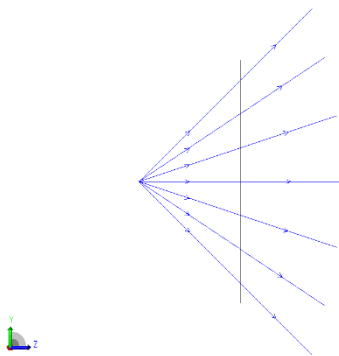


Figure 4: The half angle can extend to 180 degrees

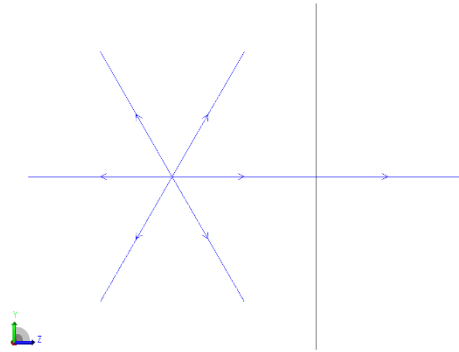


Figure 5: A negative source distance produces a diverging beam

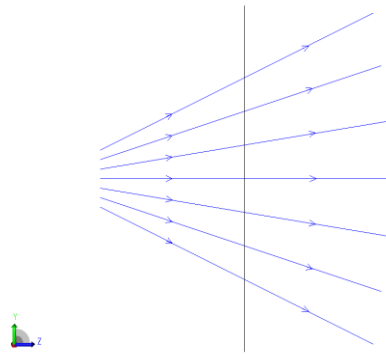


Figure 6: A positive source distance converges a beam to a point at the defined distance

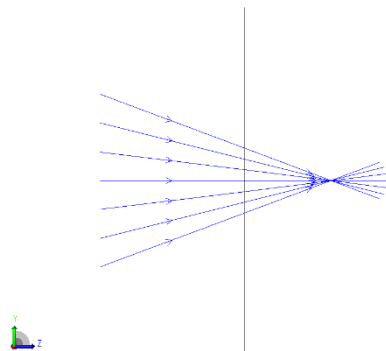
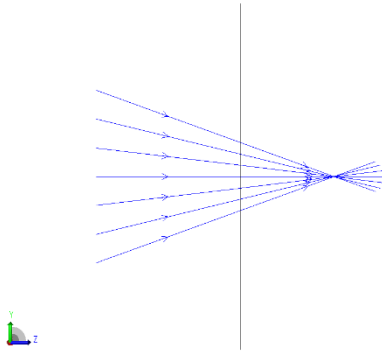


Figure 7: A beam with non-zero half height and negative half angle in tangent space also converges to a point



**** End ****